

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

MS Teams - on-line общение

- код команды **8url2s2**
- присоединитесь
- установите человеческие аватарки
 - ваше лицо
 - хорошо различимо

Правила поведения

- **занятия – мое время для вас**
- преподаватель может ошибаться
(только ваша практика и ваши знания
дают вам какие-то гарантии)
- **цените и берегите чужое время**

Как беречь время

- не отвлекайте других
 - будьте пунктуальны
 - усыпляйте ваших «тамагочи»
- правильно **спрашивайте**
привлекайте внимание (поднятой рукой)

Задавая вопрос

- задавайте вопрос
 - опускайте присказку «можно задать вопрос?»
 - «я ничего не понял» - утверждение
 - постарайтесь сформулировать вопрос
- используйте шанс получить ответ
- будьте готовы получить ответ позже (напомнить)

Лекции

- я читаю в удобном мне темпе
- конспектируйте, что успеваете
 - записывайте, зарисовывайте и т.д.
 - просьбы подождать часто игнорирую
 - слайды выкладываю
- задавать вопросы можно и нужно

Практика (лабораторные)

- для консультаций
 - демонстрации
 - разбор
 - ответы на вопросы
- отработка навыков
- проверка навыков и знаний

Кроме того, полезно помнить

- вы являетесь студентом
 - добровольно выбрали институт и специальность
 - имеете права и обязанности
- программирование
 - инструмент
 - способ мышления

ваши вопросы

**ЗАЧЕМ
МЫ С ВАМИ
ЗДЕСЬ?**

Зачем «мы» здесь

- обеспечить условия для обучения
 - информация
 - обратная связь
 - живой пример
 - пряники
 - кнут

Суть курса

- освоение разных уровней абстракции
- знакомство с «детальми реализации»
- с использованием конкретного языка

Цели курса

- навыки программирования на C++
 - знать базовый синтаксис
 - уметь пользоваться стандартной библиотекой
- практический опыт разработки
выработка стиля
- подготовить студентов к
профессиональной деятельности

Основные ресурсы

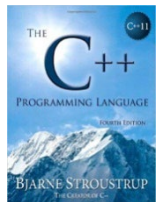
- лекции
- «Язык программирования C++. Базовый курс»
Стенли Б. Липпман, Жози Лажойе, Барбара Э. Му
- cppreference.com



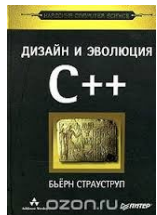
Бьерн Страуструп (книги)



Программирование. Принципы и практика с использованием C++



The C++ Programming Language (4th ed)



Дизайн и эволюция языка C++

Литература (старое и ценное)



Эккель Б. Философия C++ (2 тома)



Остерн М.Г. Обобщенное программирование и STL



Александреску А. Современное проектирование на C++

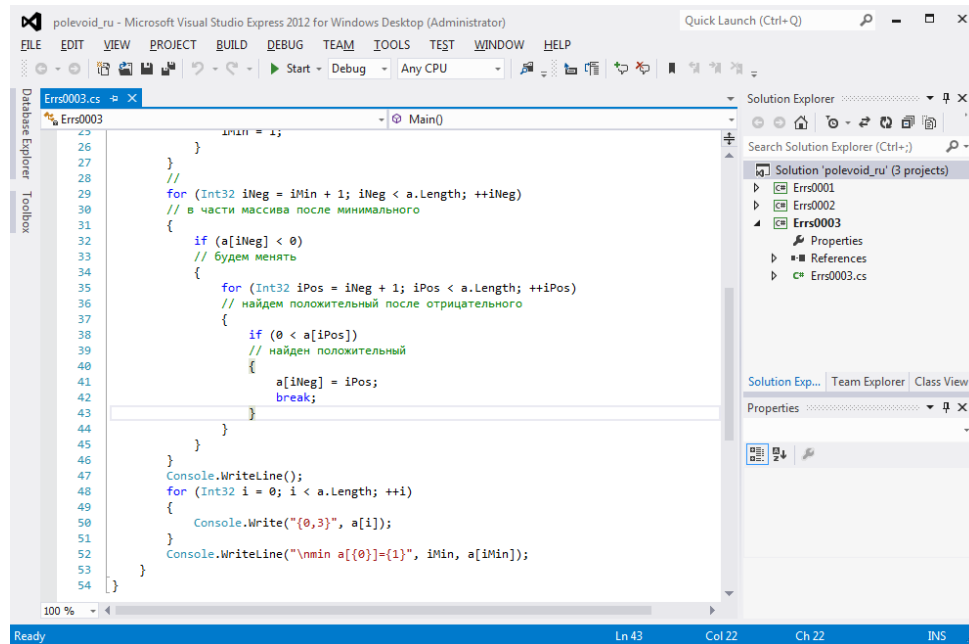
C++ через браузер

- **cpp.sh**
- **www.onlinegdb.com**
- **много других**



Integrated Development Environment (IDE)

- интегрированная среда разработки
 - текстовый редактор
 - управление файлами
 - автоматизация сборки
 - отладчик
 - и т.д. ...



ваши вопросы

Язык программирования

- определяется некоторым документом
 - стандарт, спецификация
- существуют «диалекты»
 - версии, расширения
- стандартная библиотека
- распространенные библиотеки

Примеры языков и сред

- язык: C++ (стандарты 98/03, 11, 14, 17, 20, и т.д.)
- компилятор: Microsoft, gcc, Intel, и т.д.
- IDE: MS Visual C++, Qt Creator, Eclipse, CLion, Code::Blocks

- язык: C#
- framework: .NET 1.0, .NET 2.0, ... , .NET 4.6
- реализация: Microsoft, Mono
- IDE: MS Visual Studio, MonoDevelop

Стандарт языка C++

Опубликовано

- C++98, C++03
- C++11
- C++14, **C++17**, **C++20**, C++23

ГОТОВИТСЯ

- C++26 и далее +=3

С и С++

- разные стандарты – разные языки
- родственные языки
- «плюсы» С++ не только в классах

C++ - язык общего назначения

поддерживает

- низкоуровневую разработку
- абстракцию данных
- ОО программирование
- обобщенное программирование (шаблоны)
- функциональное программирование*

Стандартная библиотека C++

- является частью стандарта
- содержит типы, константы, функции
- включает стандартную библиотеку C
- предоставляет широкий арсенал эффективных инструментов

Как оценить книгу/курс по C++?

спросите и покажите мне

- не содержит C++11 – не годится
- признаки непригодности

```
#include <io.h>
```

```
#include <math.h>
```

```
#include "stdafx.h"
```

```
#include <windows.h>
```

и т.д.

ваши вопросы

Этапы освоения

- читаем/понимаем чужой код
- переписываем чужой код
- пишем свой код
- «языковые модели» (ChatGPT)
 - могут помогать
 - не останавливайтесь на понимании

Что такое программа?

- **исполнитель** – тот, кто решает задачу
– устройство, система, человек
- **алгоритм** – конечная последовательность команд конкретного исполнителя, для достижения конкретного результата
- **программа** – формализованное описание реализации алгоритма

Зачем нужна программа?

- компьютер
 - «большой калькулятор»
 - оснащен «пищалками и звенелками»
- выполняется **машинный код**
 - данные и код (инструкции процессору) в памяти
 - писать трудно, неудобно и медленно

Программа

удобный для человека

способ управления компьютером

- текстовое описание алгоритма
- преобразуется в машинный код
- исполняется

Исходный код

- необходим для **сборки** программы
- **исходные тексты**
 - .h, .cpp (для C++)
- **файлы системы сборки**
 - CMakeLists.txt (для CMake)
 - .vcproj, .sln (файлы проектов и т.п.)

Ключевые слова ЯП

- список определяется стандартом ЯП
- не могут быть идентификаторами
- подсвечиваются в текстовом редакторе
- их надо знать (узнавать)

Ключевые слова C++

asm, auto, break, case, catch, char, class, const, continue, default, do, double, else, enum, extern, float, for, friend, goto, if, inline, int, long, new, operator, private, protected, public, register, return, short, signed, sizeof, static, struct, switch, template, this, throw, try, typedef, typeid, union, unsigned, virtual, void, volatile, while и т.д.

Комментарии

- подсказка «читателю»
- могут использоваться для генерации документации
- желательно
 - содержательные
 - полные
 - актуальные

Комментарии

- `/*` Си комментарий, многострочный `*/`
- `//` C++ комментарий, однострочный

Идентификатор (имя)

- начинается с буквы
- содержит буквы, цифры и подчеркивания
- длинна ограничена реализацией
- отличается от ключевого слова
- чувствителен к регистру

Идентификатор (примеры)

```
// имя переменной  
int32_t count(0);
```

```
// имя типа  
int32_t count(0);
```

```
// имя функции  
int main()
```

Операторы и инструкции

оператор (operator, operation)

- определяются языком
- имеет операнды и результат некоторого типа

пример: `+`, `-`, `+=`, `?` `:`, `<<`, `::` и т.д.

инструкция (statement)

- выражение, заканчивающееся `;`
- `for`, `while`, `do`, `if`, `else`,
`case`, `break`, `continue`, и т.д.

Составная инструкция, блок (composite statement, block)

последовательность инструкций,
заключенных в фигурные скобки

пример:

```
{                                     if (a < b) {  
    int a(0);                        a += 1;  
    go(a);                           }  
}
```


Выражение

- результат вычисления – значение типа T
- может содержать
 - выражения (хотя бы одно)
 - операторы (и их аргументы)
 - скобки (для установления порядка)
- элементарные выражения
 - литерал
 - переменная (в т.ч. поле)
 - вызов функции (возвращаемое значение)

Выражение (пример)

"Тестовый текст"

$a = 1$

$\text{std}::\text{abs}((a + 15) / 3.14)$

$2 * M_PI + f(g(y))$

ваши вопросы

Тип (как языковое явление)

- способ **хранения** данных в памяти
- способ **интерпретации** этих данных
- **допустимые операции** (над экземплярами)

Модель памяти

- непрерывная память
 - адрес – сдвиг относительно «начала»
 - байт – единица адресации

0



Объект

- непрерывная область памяти
- трактуется как значение типа
- именованный или неименованный

пример:

```
int num(1);
```

```
p[15]
```

Переменная

- именованный объект
- некоторого типа
- расположенный где-то в памяти
- всегда имеет значение
(но не всегда определенное)
- соответствует «экземпляру»

Типы

- **встроенные (фундаментальные, базовые)**
например: `int`, `double`, `bool` и т.д.
- **ПОЛЬЗОВАТЕЛЬСКИЕ**
 - перечисления (`enum`)
 - структуры (`struct`)
 - классы (`class`)
 - ...

Объявление (declaration)

- указывает компилятору, на сущность какого типа ссылается имя
- имя должно быть объявлено до первого использования
- часто совмещается с определением

Объявление (примеры)

```
// объявления функций
```

```
int32_t f();
```

```
bool isGood(const double val);
```

```
// объявления типов
```

```
struct Student;
```

```
class Data;
```

Определение (definition)

- «создание» сущности
- единственное для каждой сущности
- любое объявление с инициализацией является определением
- для всего, кроме функций и пространства имен заканчивается точкой с запятой

Определение

- инструкция (не является выражением)
- может содержать выражение (инициализатор)

пример:

```
int32_t a = 5;
```

выражение

```
int32_t b(a + 5 - f(a));
```

выражение

Область видимости

- объявление вводит имя в область видимости
- глобальное имя
 - объявлено вне любого блока, область видимости простирается от объявления и до конца содержащего объявление файла
- локальное имя
 - объявлено внутри некоторого блока (функция, пространство имен, класс), область видимости простирается от объявления и до конца блока

Целые типы (`int`)

- модификаторы
 - знаковость (`signed`, `unsigned`)
 - размер (`short`, `long`, `long long`)

пример:

```
int a(0);
```

```
long b(0);
```

Вещественные типы

- `float`
(32bit IEEE-754)
- `double`
(64bit IEEE-754)
- `long double`
(обычно 80bit, м.б. не IEEE-754)

Числовые литералы

- целые числа

99, 0x01AA, 011, 1L

- числа с плавающей точкой

0.5, 0.314E1, 0.3f, 0.314E1f, 0.0L

Логический тип (bool)

- принимает значения
 - true (истина)
 - false (ложь)
- результат логических выражений и сравнений

пример:

```
bool canMove(false);  
canMove = true;
```

Рекомендуемые типы

- `int` **или** `long`
- `double`
- `bool`

ваши вопросы

Выражения и приоритеты

- операнды (переменные, вызовы)
- операторы
- скобки
- таблица приоритетов (операторов)

пример:

```
a = b * (d - c);
```

```
w = h * std::cos(alph - bett);
```

```
std::cout << (a + b) << "->" << f(c);
```

Логические операторы

&&	логическое И (AND)	<code>expr && expr</code>
	логическое ИЛИ (OR)	<code>expr expr</code>
!	логическое НЕ (NOT)	<code>!expr</code>

Арифметические сравнения

==	равно	<code>expr == expr</code>
!=	не равно	<code>expr != expr</code>
>	больше	<code>expr > expr</code>
>=	больше или равно	<code>expr >= expr</code>
<	меньше	<code>expr < expr</code>
<=	меньше или равно	<code>expr <= expr</code>

Арифметические операторы

*	умножение	<code>expr * expr</code>
/	деление	<code>expr / expr</code>
%	остаток от деления (деление по модулю)	<code>expr % expr</code>
+	сложение (плюс)	<code>expr + expr</code>
-	вычитание (минус)	<code>expr - expr</code>

Присваивающие арифметические операторы

<code>*=</code>	умножение и присваивание	<code>lvalue *= expr</code>
<code>/=</code>	деление и присваивание	<code>lvalue /= expr</code>
<code>%=</code>	остаток и присваивание	<code>lvalue %= expr</code>
<code>+=</code>	сложение и присваивание	<code>lvalue += expr</code>
<code>-=</code>	вычитание и присваивание	<code>lvalue -= expr</code>

Побитовые операторы

интегральные типы и перечисления

&	И (AND)	<code>expr & expr</code>
	ИЛИ (OR)	<code>expr expr</code>
^	исключающее ИЛИ (OR)	<code>expr ^ expr</code>
~	дополнение	<code>~expr</code>
<<	сдвиг влево	<code>expr << expr</code>
>>	сдвиг вправо	<code>expr >> expr</code>

Присваивающие побитовые операторы

<code>&=</code>	И (AND)	<code>lvalue &= expr</code>
<code> =</code>	ИЛИ (OR)	<code>lvalue = expr</code>
<code>^=</code>	исключающее ИЛИ (OR)	<code>lvalue ^= expr</code>
<code><<=</code>	сдвиг влево	<code>lvalue <<= expr</code>
<code>>>=</code>	сдвиг вправо	<code>lvalue >>= expr</code>

Инкремент и декремент

- **инкремент**

`++lvalue` (возвращает новое значение)

`lvalue++` (возвращает старое значение)

- **декремент**

`--lvalue` (возвращает новое значение)

`lvalue--` (возвращает старое значение)

ваши вопросы

Управление исполнением программы

- выбор (`if`, `else`, `switch`)
- цикл (`for`, `while`, `do while`)
- переход (`break`, `continue`, `goto`)
- функции

Инструкции выбора

- `if` (условие)
инструкция
- `if` (условие)
инструкция
`else`
инструкция
- `switch` (условие)
инструкция

if-инструкция

```
if (x) //...
```

означает

```
if (0 != x) //...
```

```
if (!x) //...
```

означает

```
if (0 == x) //...
```

if-инструкция (пример)

```
if (0 == isReady) {  
    // основная работа  
} else {  
    // обработка дополнительных случаев  
}
```


switch-инструкция

- значение сопоставляется с набором констант
- можно заменить набором if-инструкций

switch-инструкция (пример)

```
switch (val) {  
  case 1:  
    fA();  
    break;  
  case 2:  
    fB();  
    break;  
  default:  
    onError();  
    break;  
}
```

Инструкции циклов

for (инициализатор; условие; выражение)

инструкция

for (объявление : интервал)

инструкция

while (условие)

инструкция

do

инструкция

while (условие);

Инструкции перехода

- `break`
 - завершить инструкцию цикла
 - завершить `switch`-инструкцию
- `continue`
 - перейти к следующей итерации цикла
- `goto`
 - не использовать

Объявление в for-инструкции

- переменную можно объявить в части инициализации for-инструкции
- область видимости простирается до конца for-инструкции

пример:

```
for (int i = 0; i < n; i += 1)
```

Пример (for-инструкция)

```
for (int i = 0; i < n; i += 1) {  
    // основная работа  
}
```

```
for (int i = n - 1; 0 <= i; i -= 2) {  
    // основная работа  
}
```

Пример (while-инструкция)

```
bool isReady(false);  
while (!isReady) {  
    // основная работа  
}
```

«Бесконечные» циклы

- «в природе» не встречаются
- явно вводите и проверяйте управляющую выполнением цикла переменную

ваши вопросы

Пустой тип (void)

`void`

в качестве возвращаемого значения, для указания, что функция ничего не возвращает

`void*`

указатель на объекты неизвестного типа

Функция (пример)

```
int main() {  
    // . . .  
    return 0;  
}
```

Функция

- изолированный фрагмент кода
- решает некоторую задачу
- предназначен для повторного использования
 - оформлен по правилам языка
 - может принимать параметры (вход)
 - может возвращать некоторое значение (выход)

Объявление функции

- тип возвращаемого значения
T или `void` (ничего не возвращается)
- имя функции
- список параметров
 - может быть пустым
 - пары «тип значение» через запятую (список)

Объявление функции (примеры)

```
void f(T1 a, const int b);
```

```
double myPi();
```

```
int max(const int lhs, const int rhs);
```

Определение функции

- объявление + тело
- типы аргументов объявления и определения должны совпадать
- рекомендуется поддерживать соответствие имен аргументов (могут не совпадать)

Определение функции

(примеры)

```
int main() {  
    //...  
    return 0;  
}
```

```
int min(const int& lhs, const int& rhs) {  
    return (lhs < rhs) ? lhs : rhs;  
}
```


Аргумент (параметр)

- семантика локальной переменной
- инициализируется в точке вызова
- область видимости = тело функции

Передача аргументов

- по значению
- по ссылке

пример:

```
int min(int lhs, int rhs) {  
    return (lhs < rhs ) ? lhs : rhs;  
}
```

Ссылка (reference)

- тип `T&` – тип «ссылка на `T`»
- альтернативное имя переменной
- инициализируется при определении

пример:

```
T var = 0;      // переменная
```

```
T& ref = var;  // ссылка на var
```

Аргумент ссылка (пример)

```
void swap(int& lhs, int& rhs) {  
    int newLhs = rhs;  
    rhs = lhs;  
    lhs = newLhs;  
}
```

Возвращаемое значение

- задается выражением в инструкции `return`
- должно быть, если не `void`-функция
- `void`-функция может не иметь `return`
- неименованная временная переменная
- должно быть сохранено в точке вызова, если нужно далее

Возвращаемое значение (пример)

```
void empty() {  
    // ...  
    return;  
}
```

```
void empty() {  
    // ...  
}
```

```
int empty() {  
    int res = 0;  
    // ...  
    return res;  
}
```

```
...  
f = empty();
```

Вызов функции

- **ВЫЗОВ**
 - обращение по имени
 - перечисление аргументов
- **аргумент**
 - фактически передаваемое значение параметра
 - определен в точке вызова
 - передается в зависимости от типа параметра

ваши вопросы

#include <cmath>

(примеры функций)

<code>std::abs</code>	модуль
<code>std::min</code>	минимум
<code>std::max</code>	максимум
<code>std::exp</code>	е в заданной степени
<code>std::log</code>	натуральный логарифм
<code>std::pow</code>	возведение в степень
<code>std::sqrt</code>	квадратный корень

#include <cmath>

(примеры функций)

<code>std::sin</code>	синус
<code>std::cos</code>	косинус
<code>std::tan</code>	тангенс
<code>std::atan2</code>	арктангенс (с учетом квадранта)

И Т.Д.

Математические константы

(примеры)

M_E	2.71828182845904523536
M_PI	3.14159265358979323846
M_PI_2	1.57079632679489661923
M_2_PI	0.636619772367581343076
M_SQRT2	1.41421356237309504880

и т.д.

Как использовать КОНСТАНТЫ (до C++20)

```
// win-специфика  
#define _USE_MATH_DEFINES  
#include <cmath>
```

Как использовать КОНСТАНТЫ (начиная с C++20)

```
#include <numbers>
```

```
std::numbers::pi
```

ваши вопросы

ваши вопросы

Как правильно делать задания

- самостоятельно
- своевременно
- в имеющемся у вас ресурсе времени
- до возможного результата
- ищите «ресурс»
(спросить/прочитать/посмотреть)

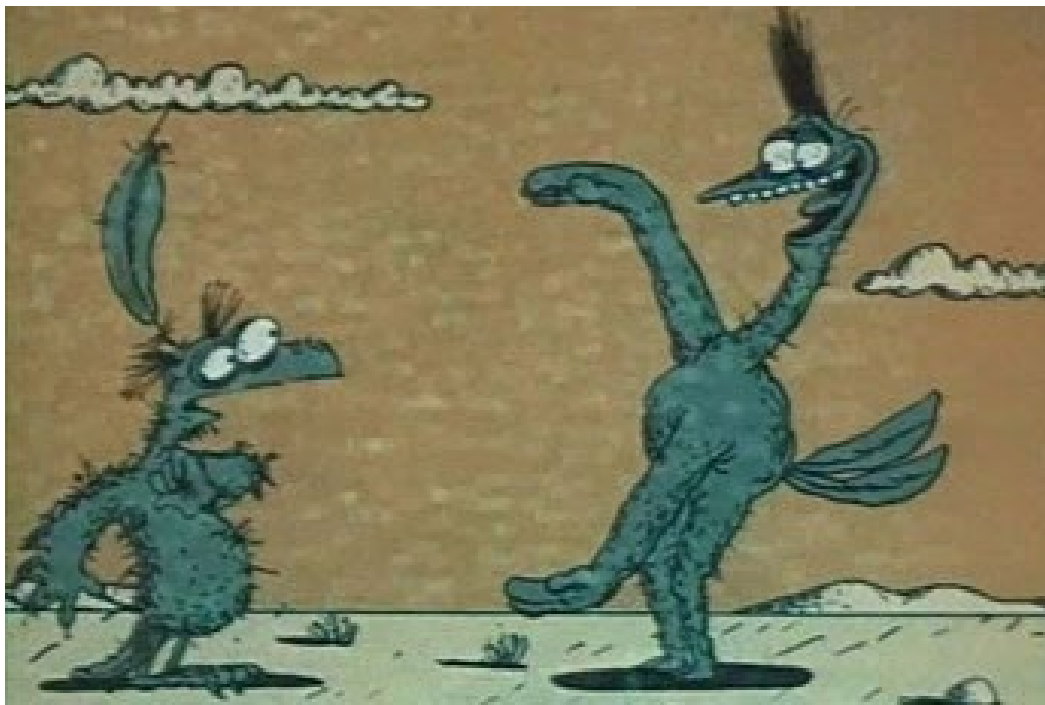
На что обратить внимание

- у всех разный стартовый уровень
- отслеживайте собственный прогресс (а не сравнивайте себя с другими)
- берите от занятий и заданий по максимуму
- учитесь учиться
- больше пишите

«Лучше день потерять,
потом за пять минут долететь!»



11.09.2025



2025-осень-Полевой

106